AR525: Reinforcement Learning in Robotics

Final Presentation — MC-PILOT: Reproduction, Analysis & Extension

Group-3

Rishang Yadav, Bhumika Gupta, Aarya Agarwal, Yajesh Chandra

April 27, 2026



Outline

1 Problem & Motivation

2 Background: MC-PILOT

3 Our Contributions

4 Two Convergence Rules

5 Study 1: Baseline & Elevated Heights

6 Study 2: Multi-Arm Generalisation

7 Study 3: Noise Models

8 Study 4: Wind Robustness

9 Study 5: Vision Pipeline

10 Results Summary

11 Key Contributions

12 Future Work

Problem & Motivation

Pick-and-Throw Operations

The Task

- Robot arm grasps a ball and **throws** it into a target bin at distance 0.75–2.4 m
- Ball follows a parabolic + drag trajectory after release — physics is *known but noisy*
- Gap between **commanded** and **delivered** velocity is random and arm-dependent
- Naive approach (model-free RL): needs **100s–1000s** of real throws

Key Question

*Can a robot learn to throw accurately from just ∼**10 attempts?***

Why it's hard

- Precise gripper timing at high speed
- Object mass & drag uncertainty
- Physical wear limits real trials

Background: MC-PILOT

Algorithmic Lineage

Algorithm	Year	Key Idea	Data Needed
Model-free RL (DQN, PPO)	2013–20	Learn from reward only	∼1000s trials
PILCO	2011	GP dynamics + analytic gradient	∼50 trials
MC-PILCO	2022	Monte Carlo gradient through GP	∼20–50 trials
MC-PILOT	**2025**	Single-shot throwing adaptation	**∼5+10 throws**

Why Gaussian Processes for dynamics?

- Exact uncertainty quantification — policy gradient knows *model confidence*
- Bayesian: posterior updates analytically with each new throw
- Data-efficient — 5 exploration throws suffice to bootstrap learning

MC-PILOT: Two Key Modifications over MC-PILCO

Modification 1 — Single-Shot Policy

- Standard MC-PILCO fires the policy at *every* timestep (suitable for balancing tasks)
- Throwing: policy fires **once** → choose release speed → ball is in free flight
- Policy: $\pi_\theta(P) \to v$ State: $\tilde{x} = [x, y, z, v_x, v_y, v_z, P_x, P_y] \in \mathbb{R}^8$

Modification 2 — Gripper Delay Estimation

- Real gripper doesn't open instantly ⇒ random delay ⇒ velocity undershoot
- MC-PILOT fits delay distribution $t_d \sim \mathcal{U}(a, b)$ via Bayesian Optimisation on hardware
- Enables deployment on **real hardware** (Franka Panda + real bin)

Paper Claim (Verified)

$\sim$10 learning throws → near-100% hit rate. Model-free baselines need 100× more data.

Our Contributions

What We Set Out To Do

We replaced Gazebo/ROS with a **pure Python + PyBullet** environment (no ROS required for ball physics).

Goal	Status
Reproduce MC-PILOT baseline ($z = 0.5\,\text{m}$, Franka Panda)	✓ Done
Diagnose & fix silent implementation bugs	✓ Done
Generalise to elevated release heights ($z = 1.0, 1.5, 2.0\,\text{m}$)	✓ Done
Generalise to 3 different robot arms	✓ Done
Model arm-side noise (slip, jitter, salt-pepper)	✓ Done
Wind conditions (constant, gusts, turbulence)	✓ Done
OpenCV + YOLO bin detection $\rightarrow$ autonomous targeting	✓ Done

Why PyBullet for RL Training?

Aspect	PyBullet Advantage	Gazebo Limitation
Sample efficiency	High rollout throughput (fast step + reset)	Lower throughput due to overhead
Policy learning	Better exploration of release timing & velocity	Fewer episodes $\rightarrow$ slower convergence
Training loop	Tight Python RL loop integration	ROS-based loop adds latency
Episode reset	Fast environment reinitialization	Slower world reset/reload

Two Convergence Rules

Rule 1 — Stratified Exploration

The Problem with Random Exploration

- Random exploration with 5 throws can accidentally draw 4 *low-speed* throws
- **Config B, seed=1,** $z = 1.0\,\mathbf{m}$: GP has no data on high-speed dynamics $\Rightarrow$ **0/10 hits**

Fix: Stratified Band Sampling

Divide $[0, u_M]$ into N_{exp} equal bands; sample exactly one throw per band:

$$v_i \sim \mathcal{U}\left(\frac{(i-1)\,u_M}{N_{\text{exp}}},\ \frac{i\,u_M}{N_{\text{exp}}}\right), \quad i = 1, \ldots, N_{\text{exp}}$$

Guarantees full speed-range coverage regardless of random seed. Zero extra hardware trials.

Evidence

Config B-random seed=1 $\rightarrow$ **0/10**. Same config, stratified $\rightarrow$ **10/10**.

Rule 2 — RBF Lengthscale Must Match Target Range

RBF Sensitivity Formula

$$S(d, \ell_s) = \exp\left(-\frac{d^2}{2\ell_s^2}\right)$$

$S \to 1$: policy outputs *identical* speed for all targets. $S \to 0$: targets are well discriminated.

Paper uses $\ell_s = 1.0\,\text{m}$ everywhere. For **Config E** (in elevated target range 0.35 m):

$$S = \exp\left(-\frac{0.35^2}{2 \times 1.0^2}\right) = 0.941 \quad \Rightarrow \text{policy is } \textbf{constant} \text{ for all targets}$$

Scaling Rule

$$\ell_s^* \approx 0.15 \times \Delta_{\text{range}}$$

This maintains $S \approx 0.066$ — the same discriminability level as the working configs.

Evidence

Config E failed **4 consecutive iterations** with $\ell_s = 1.0\,\text{m}$. Changed to $\ell_s = 0.15\,\text{m} \to$ **10/10 hits immediately**.

Study 1: Baseline & Elevated Heights

Baseline Result: 5/5 Hits in 10 Total Throws

Throw	Phase	Error	Hit?
1–5	Exploration	0.02–1.09 m	—
6	Trial 1	8.5 cm	✓
7	Trial 2	3.3 cm	✓
8	Trial 3	2.3 cm	✓
9	Trial 4	1.7 cm	✓
10	Trial 5	0.8 cm	✓

Cost trajectory:

$0.74 \xrightarrow{\text{Trial 1}} 0.009 \xrightarrow{\text{Trial 5}} 0.001$

Physical Meaning

GP learns the speed-to-range curve from **5 random throws**. Policy optimisation (1500 gradient steps/trial) finds the exact release speed per target. By trial 5, errors are **sub-centimetre**.

Config A: $z_{rel} = 0.5\,m$, $\ell_M = 1.75\,m$, *Franka Panda, no noise.*

Generalising to Elevated Release Heights

Engineering challenges:

- KUKA iiwa7 joint limits cap EE speed at ∼2.5 m/s; throw needs 3.5 m/s → *decoupled*: ball velocity set via `resetBaseVelocity`, arm is cosmetic
- EE cannot reach $z = 1.5$ m from ground → mount arm on pedestal: $z_{\text{base}} = z_{\text{rel}} - 0.5$ m

Config	z_{rel}	Exploration	Hit rate	Mean error	Cost trajectory
A	0.5 m	Random	100% (5/5)	33 mm	0.740 → 0.001
B	1.0 m	Random	100% (10/10)	20 mm	0.882 → 0.005
B-Strat	1.0 m	Stratified	100% (10/10)	20 mm	0.811 → 0.003
C-Strat	1.5 m	Stratified	100% (10/10)	14 mm	0.809 → 0.085
D-Strat	2.0 m	Stratified	100% (10/10)	16 mm	0.827 → 0.024
E-Strat5	0 m	Stratified, $\ell_s = 0.15$	80% (12/15)	69 mm†	− − − → 0.031

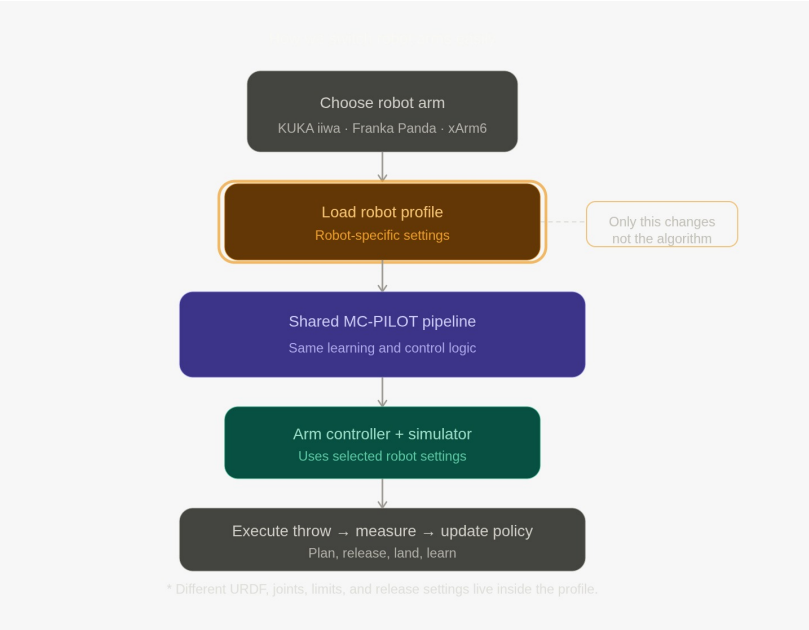
† *E-Strat5: first 5 throws still converging (mean 155 mm); throws 6–15 post-convergence achieved 100% hit rate with mean error 26 mm. Config A early-stopped at step 1204 on Trial 5; trial errors: 85 → 33 → 23 → 17 → 8 mm.*

Study 2: Multi-Arm Generalisation

Three Robot Arms: Franka Panda, KUKA iiwa7, xArm6

Key Architectural Insight — Decoupled Design

- RL trains on **ball physics only** (NumPy ballistic simulator)
- Joint velocities calculated depending on the arm to set ball velocity at release.
- Joint tracking error is substituted by our own error; matching MC-PILOT.



Cross-Arm Results

Unified Robot Throwing Summary

[Robot (w/image & name)]	[Hit Rate]	[Mean Error]	[Median Error]	[Max Error]	[Avg Speed]	[Release time]
 Franka Panda	Hit Rate 100% (5/5 throws)	Mean Error 0.50 cm	Median Error 0.48 cm	Max Error 0.74 cm	Avg Speed 1.404 m/s	Release time 0.600 s
 KUKA iiwa	Hit Rate 100%	Mean Error 1.60 cm	Median Error 1.41 cm	Max Error 2.74 cm	Avg Speed 2.186 m/s	Release time 0.600 s
 xArm 6	Hit Rate 100%	Mean Error 2.44 cm	Median Error —	Max Error 3.30 cm	Avg Speed 1.334 m/s	Release time 0.500 s

Study 3: Noise Models

Arm-Side Stochasticity: Making RL Non-Trivial

Why noise matters

- Without noise: a ballistic inverter solves the task in one step — **RL adds no value**
- With noise: policy must learn to compensate — **RL is necessary**

Two learnable noise types:

- *VelocitySlipNoise:* $v_{\text{actual}} = (1-\alpha)\, v_{\text{cmd}} + \mathcal{N}(0, \sigma^2)$ Systematic undershoot; policy must learn to command $v/(1-\alpha)$
- *Salt-and-Pepper:* velocity component randomly replaced by a spike; models encoder glitches

Key Theorem (Proven)

Zero-mean **symmetric** noise does **not** shift the optimal policy:

$$\nabla_{v^*} J_{\text{aware}} = \nabla_{v^*} J_{\text{naive}}$$

Only **biased** noise (slip, jitter) is learnable and compensable. Paper's gripper delay model is biased — which is why it works on hardware.

Noise Results: Aware Policy vs. Naive Policy

Noise type	Noise level	Noise-aware training			Naive (unaware) training		
		Hit rate	Mean error	Final cost	Hit rate	Mean error	Final cost
Slip noise ($\sigma = 0.04$ fixed)							
Slip	$\alpha = 0.05$	100%	0.0260 m	0.00700	100%	0.0682 m	0.00349
Slip	$\alpha = 0.10$	100%	0.0450 m	0.00672	100%	0.0578 m	0.00339
Slip	$\alpha = 0.15$	100%	0.0359 m	0.00623	25%	0.1242 m	0.00355
Slip	$\alpha = 0.20$	100%	0.0219 m	0.01097	0%	0.1881 m	0.00404
Salt-and-Pepper noise (spike scale $s = 0.30$ fixed)							
Salt-and-Pepper	$p = 0.05$	100%	0.0292 m	0.01605	75%	0.0439 m	0.00410
Salt-and-Pepper	$p = 0.10$	75%	0.1104 m	0.02381	50%	0.0768 m	0.00410
Salt-and-Pepper	$p = 0.15$	50%	0.0914 m	0.03143	25%	0.1012 m	0.00345
Gaussian velocity bias noise (VelocityBiasNoise)							
Gaussian	$\sigma = 0.15$	30%	0.141 m	—	20%	0.151 m	—

Hit rates colour-coded: 100% · 75% · 50% · 30% · 25% · 20% · 0%. Final cost not reported for Gaussian condition. Mean errors for Gaussian converted from cm to m for consistency.

Study 4: Wind Robustness

Wind Modelling & Environmental Disturbances

Problem Context

- The standard MC-PILOT model assumes **calm, still air**, limiting reliability in semi-outdoor deployments where modest wind can shift lightweight payloads.

Experimental Setup

- Extended the physics simulator with a **wind force term** integrated into the aerodynamic drag equation.
- Evaluated against three dynamic wind conditions: **constant wind**, **random gusts** (step changes in w), and **turbulence** (Gaussian noise on w).
- Compared a standard **Wind-Blind** model (8D state) against a **Wind-Aware** model (10D state with explicit wind input) to determine if the GP implicitly absorbs disturbances.

Empirical Findings: Wind-Aware vs. Wind-Blind

Key Results

- **Accuracy Degradation:** Constant wind degrades base performance (Hit rate dropped from 73% to 47% under strong wind).
- **Implicit Absorption:** Surprisingly, the **Wind-Blind (8D)** model consistently **outperformed** the Wind-Aware (10D) model across all dynamic conditions.
- For example, in Turbulence (W3), the blind baseline achieved a 60% hit rate compared to the aware model's 47%.

Conclusion

Expanding the state space to 10D increased the learning complexity beyond what the GP could efficiently sample within limited trials. The 8D model was robust enough to implicitly absorb the wind disturbances as a bias term.

Performance Analysis: Hit Rates

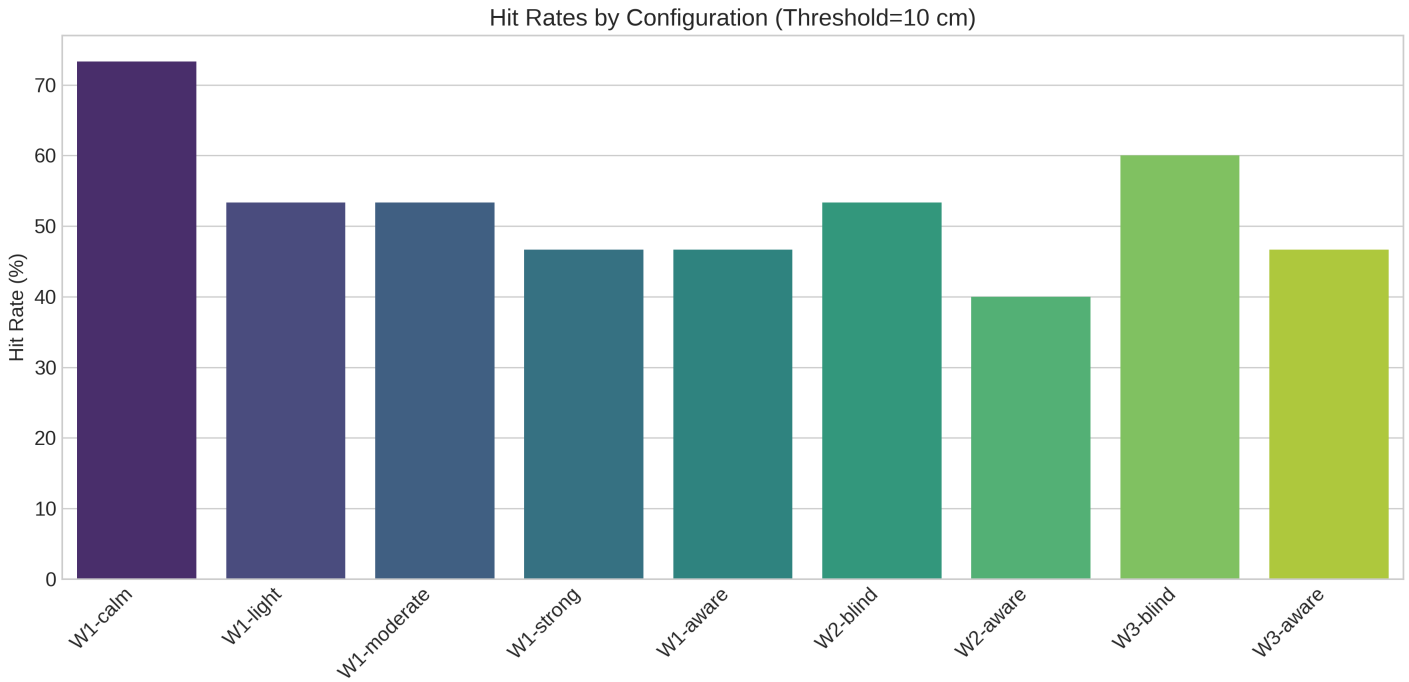


Figure: Comparison of Hit Rates (landing within 10cm threshold) across configurations.

Performance Analysis: Error Variance

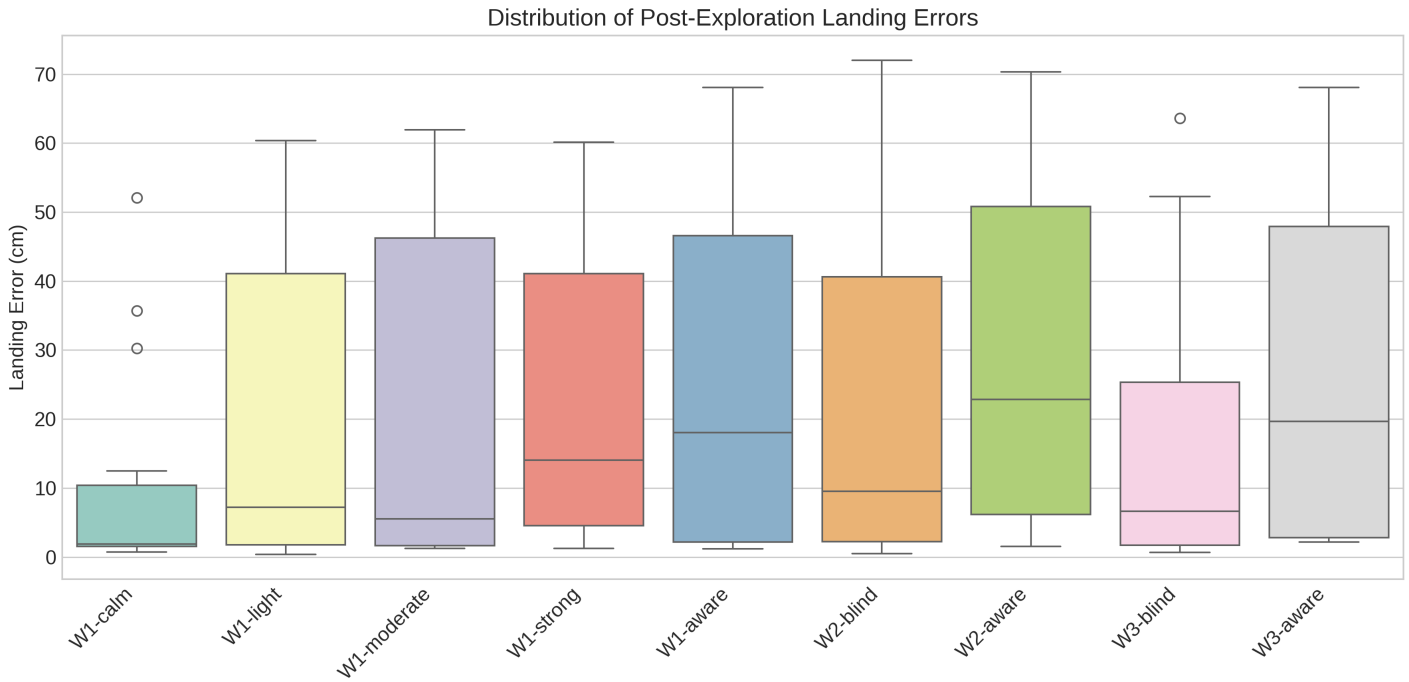


Figure: Landing Error Variance and Distribution for post-exploration trials.

Performance Analysis: Learning Convergence

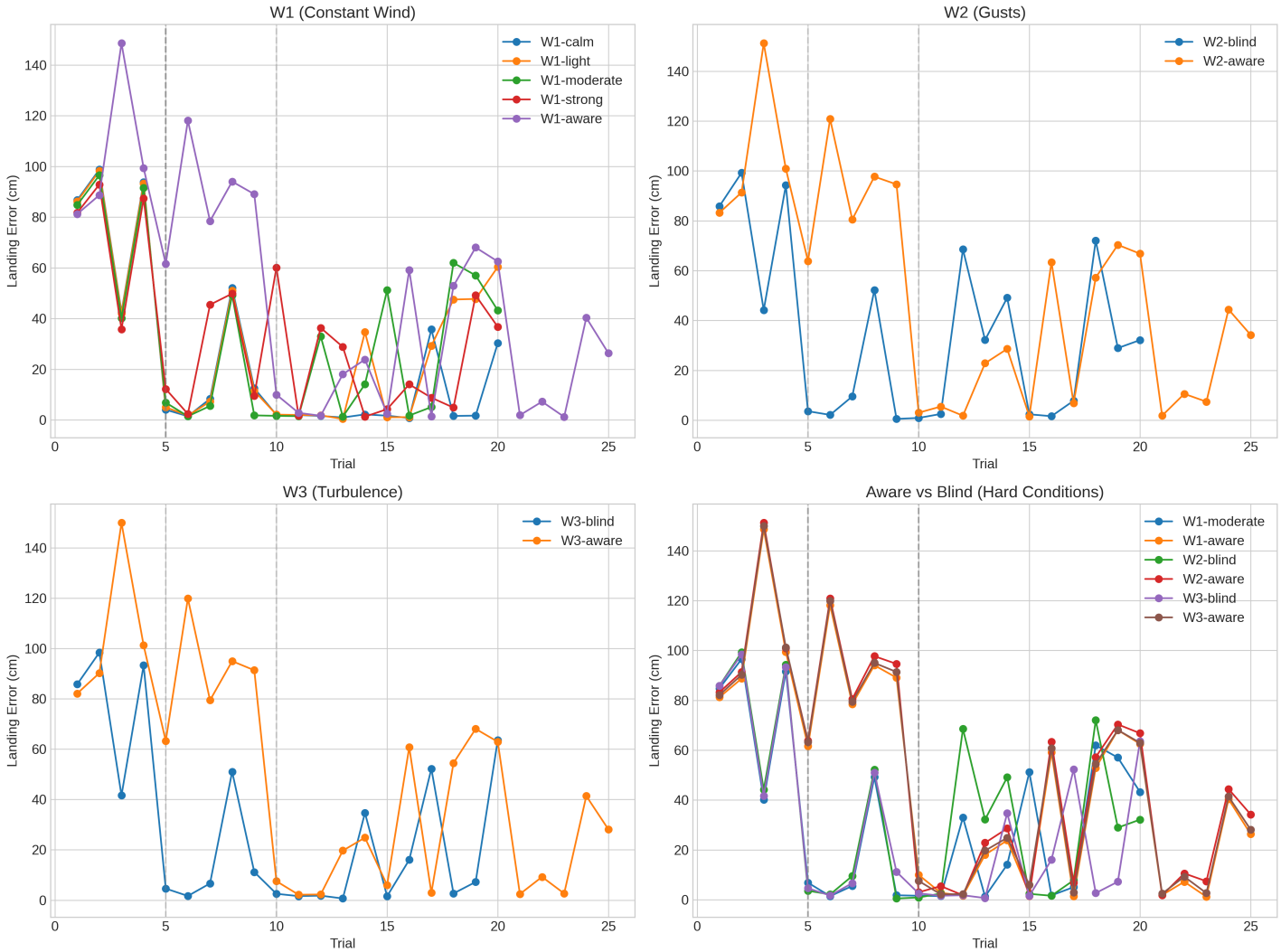
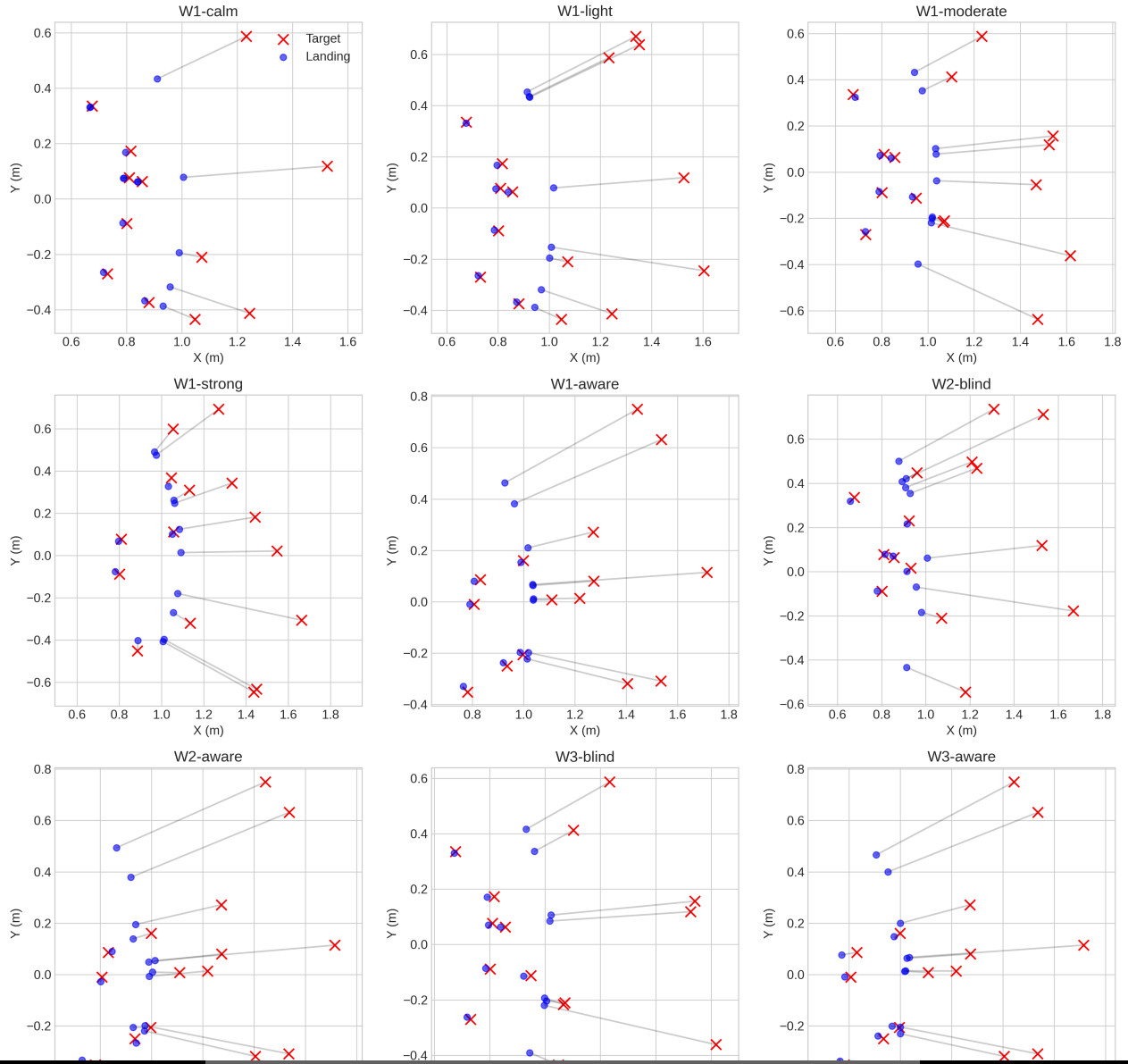


Figure: Physical mean error convergence over successive optimization trials.

Performance Analysis: Target Landing Distributions



Study 5: Vision Pipeline

OpenCV + YOLO Bin Detection

Problem

So far, target coordinates are given numerically. For real deployment, the robot must *detect* the bin visually.

OpenCV (Geometric)

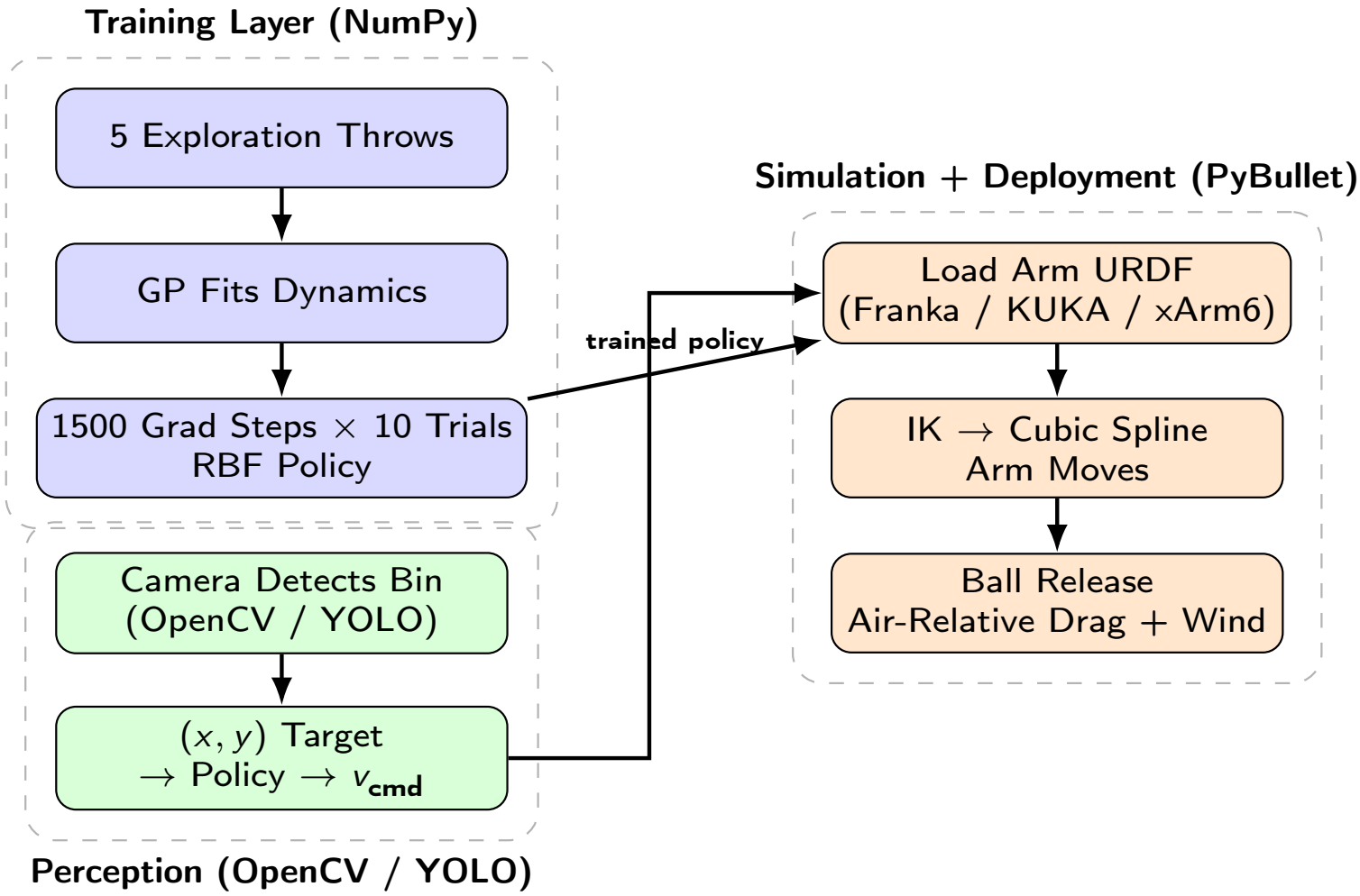
- Depth camera on arm base (PyBullet RGB-D)
- Segment green bin by HSV threshold $\rightarrow$ contour $\rightarrow$ centroid
- Depth back-projection: pixel $(u, v, d) \rightarrow$ world (x, y)
- Bias correction: camera sees near face of bin $\rightarrow$ shift by $2R/\pi$

YOLO (Learning-Based)

- Synthetic dataset: 800 train / 200 val images rendered in PyBullet
- YOLOv8n trained on rendered red sphere target
- Bounding box centroid + depth $\rightarrow$ 3D target estimate
- Robust to occlusion and lighting variation

Both pipelines feed detected (x, y) into policy $\rightarrow$ throw command. Bin is **hollow** $\rightarrow$ ball physically enters bin. We have also tested the algorithm on different positions of camera.

Full System Architecture



Results Summary

Complete Results Summary

Study	Configuration	Hit Rate	Key Insight
Baseline	$z = 0.5$ m, Franka, no noise	5/5 (100%)	Cost 0.74 $\rightarrow$ 0.001 in 5 trials
Elevated	$z = 1.0$ m, stratified	10/10	Pedestal + stratified essential
Elevated	$z = 1.5$ m, stratified	10/10	ℓ_M, T scaled with height
Elevated	$z = 2.0$ m, stratified	10/10	10 consecutive hits
Multi-arm	Franka/KUKA/xArm6	100% each	Arm-agnostic RL validated
Noise	Slip $\alpha = 0.20$, aware	100% vs 0% naive	Infinite relative improvement
Noise	Salt-pepper $p = 0.05$, aware	100% vs 75% naive	Consistent advantage
Wind	W1 calm, blind	73%	GP learns physics well
Wind	W3 turbulence, blind	60% vs 47% aware	Implicit beats explicit
Vision	OpenCV + YOLO	End-to-end	Detect $\rightarrow$ throw in one loop

Key Contributions

Key Scientific Contributions

Three things the paper never stated that are necessary for the algorithm to work:

1. **Stratified Exploration** — random seed can produce zero-coverage exploration; stratified band partitioning guarantees learning regardless of seed
2. **RBF Lengthscale Rule** — $\ell_s \approx 0.15 \times \Delta_{\text{range}}$; at $\ell_s = 1.0\,\text{m}$ and range $= 0.35\,\text{m}$, policy outputs a constant speed for *all* targets
3. **Zero-Mean Noise Invariance (formally proven)** — symmetric Gaussian noise cannot be compensated by RL; only *biased* noise (slip, timing jitter) is learnable. Paper's gripper delay model is biased, which is why it works on hardware.

Future Work

Future Work

- **Hardware validation** — run trained policies on a real robot arm; measure PyBullet-to-real transfer degradation and where the 5–10% drag discrepancy compounds with real-world effects
- **Wind-robust policy** — randomise wind per rollout during training so a single policy handles a *distribution* of wind conditions, not one fixed vector
- **End-to-end YOLO deployment** — vision detection $\rightarrow$ policy $\rightarrow$ throw $\rightarrow$ physical landing in bin, fully in one hardware loop
- **Multi-target sequences** — policy throws to a sequence of bins without retraining; requires online target switching
- **Adaptive noise estimation** — estimate slip/jitter parameters *online* via Bayesian Optimisation (paper's approach) rather than requiring them upfront; integrate with the wind-aware pipeline

Thank You!